MoMo SMS Transactions API — Project Report
Course: Enterprise Web Development
Team Name: Les Genies
Submission Date: May 2026

Github repository:https://github.com/Emmanuella2006/ewd-teamsetup-and-project-planning_lesgenies
Team task sheet:
https://docs.google.com/spreadsheets/d/1L2OiwxI-nxn_As7Y8DPjAHs9vX9kgVKgEb-Q1pnWVlY/edit?usp=sharing

Table of Contents

1. Introduction to API Security

An API (Application Programming Interface) is the communication layer that allows different software systems to exchange data. In the context of this project, the MoMo SMS Transactions API exposes mobile money records to client applications such as web and mobile apps. Because financial transaction data is sensitive, securing the API is not optional, it is a core requirement.

API security covers several concerns:

Authentication confirms the identity of whoever is making a request. This project implements HTTP Basic Authentication as a first layer of access control, requiring a valid username and password on every request.

Authorization determines what an authenticated user is allowed to do. In this implementation all authenticated users share the same access level, but in production systems, different roles (read-only, admin, auditor) would restrict what each user can perform.

Input validation protects the server from malformed or malicious data. Every POST and PUT request should be checked before it is stored. Accepting arbitrary input without validation opens the door to injection attacks, data corruption, and application crashes.

Rate Limiting prevents abuse by capping the number of requests a single client can make in a given time window.
This project focuses on authentication as the primary security mechanism.

2. API Documentation

2.1 Overview

Property	**Value**
Base URL	**http://localhost:8080**
Protocol	HTTP (HTTPS recommended for production)
Data Format	JSON
Authentication	HTTP Basic Authentication
Server	Python **http.server** module

2.2 Transaction Object

All endpoints produce and consume the following JSON structure:

Field	**Type**	**Description**
id	string	Unique identifier for the transaction
type	string	Transaction type: **payment**, **incoming**, or **outgoing**
amount	number	Transaction amount as float
sender	string	Name or identifier of the sending party
receiver	string	Name or identifier of the receiving party
timestamp	string	ISO 8601 datetime string, e.g. 2026-05-27T12:00:00Z

2.3 Authentication

All endpoints require HTTP Basic Authentication. The server validates credentials on every request before processing it.

Credentials:

Field	Value
Username	**admin**
Password	password123

How to include a credential in a request:

Option 1:
curl -u admin:password123 http://localhost:8080/transactions

Option 2: build the header manually. Base64-encode **admin:password123** to get YWRtaW46cGFzc3dvcmQxMjM=, then include:
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

How the server validates credentials (from api/server.py):

The **check_auth()** method reads the **Authorization** header, strips the **Basic** prefix, base64-decodes the remainder, and splits on **:** to extract the username and password. These are compared against the hardcoded **VALID_USERNAME** and **VALID_PASSWORD** constants. If the header is missing, malformed, or the credentials do not match, the method returns **False** and the server sends a **401** response.

Unauthorized response:
{"error": "Unauthorized"}

The 401 response also includes the header WWW-Authenticate: Basic realm="MoMo API", which prompts browsers to display a login dialog.

2.4 Endpoints
GET/transactions

Returns a list of all transactions records currently held in the data store.

Method: GET
URL: /transactions
Auth Required: Yes
Request body: None
Request example:
curl -X GET http://localhost:8080/transactions \-u admin:password123

Error Codes

Status Code	Description
200 OK	Succes-returns array of all transactions
401 Unauthorized	Missing or invalid credentials

GET/transactions/{id}

Returns a single transactions identified by its unique ID

Method: GET
URL: /transactions/{id}
Auth required: Yes
Request body: None

Path parameter:

Parameter	Type	Description
id	string	Unique ID of the transaction

Request example:
```
curl -X GET http://localhost:8080/transactions/1 \
  -u admin:password123
```

Error codes:

Status Code	Description
200 OK	Success: returns the matching transaction
401 Unauthorized	Missing or invalid credentials
404 Not Found	No transaction exists with the given ID

POST /transaction

Adds a new transaction to the data store. The new record is appended to DATA_STORE and indexed in DICTIONARY_STORE for fast lookup.

Method: POST

URL: /transaction
Auth required: Yes
Content-Type: application/json

Request Example:
curl -X POST http://localhost:8080/transactions \
  -u admin:password123 \
  -H "Content-Type: application/json" \
  -d '{"id": "2", "type": "incoming", "amount": 1000.0, "sender": "Charlie", "receiver": "Alice", "timestamp": "2026-05-27T14:00:00Z"}'

Error codes:

Status Code	Description
201	Transaction successfully added
400 Bad Request	Malformed JSON or missing required fields
401 Unauthorized	Missing or invalid credentials

PUT/ transaction/{id}
Updates one or more fields of an existing transaction. Fields not included in the request body remain unchanged. The server updates both DATA_STORE and DICTIONARY_STORE to keep them in sync.

Method: PUT
URL: /transactions/{id}
Auth required: Yes
Content-Type: application/json

Path parameter:

Parameter	Type	Description
id	string	Unique ID of the transaction to update

Request Example:
curl -X PUT http://localhost:8080/transactions/1 \
  -u admin:password123 \
  -H "Content-Type: application/json" \
  -d '{"amount": 750.0, "type": "outgoing"}'

Error Code:

Status Code	**Description**
200 OK	Transaction updated - full record returned
401 Unauthorized	Missing or invalid credentials
404 Not Found	No transaction exists with the given ID

DELETE /transactions/{id}

Permanently removes a transaction from the data store. The record is deleted from both DICTIONARY_STORE and DATA_STORE.
Method: DELETE
URL: /transactions/{id}
Auth required: Yes
Request body: None

Path parameter:

Parameter	Type	Description
id	string	Unique ID of the transaction to delete

Request example:
curl -X DELETE http://localhost:8080/transactions/1 \-u admin:password123
Error Codes:

Status Code	Description
200 OK	Transaction successfully deleted
401 Unauthorized	Missing or Invalid credentials
404 Not Found	No transaction exists with the given ID

Error Codes Reference

This table summarizes all HTTP status codes the API can return and what they mean

Status code	When it occurs

200 OK	GET, PUT, DELETE succeeded
201 Created	POST succeeded, and a new record was stored
400 Bad Request	Request body is not a valid JSON or is missing required fields
401 Unauthorized	The authorization header is missing, malformed, or the credentials are wrong
404 Not Found	The transaction ID does not exist, or the route is not recognised
500 Internal Server Error	An unexpected server-side error occured

3. Data Structures & Algorithms

3.1 Search Approaches

Data is only useful if you can find it fast. Looking at 1691 transactions parsed from the MoMo SMS data, the first question that comes to mind is how to find a specific record without going through all of them. This is exactly what I am going to discuss, which is implementing two search approaches and tracking which one actually performs better.

1. Linear Search

This is a more straightforward approach. It started at the first transaction and checked each one until the record with the ID that the user is looking for was found. Therefore, if the record is near the end of the list, the user might end up scanning almost everything before getting there. So, the logic works, but the farther the record is, the longer it takes. And it is what is called O(n), and it implies that the time it takes grows with data.

2. Dictionary Lookup

Here, the list got converted into a dictionary where each transaction ID is a key pointing directly to its record. Therefore, finding transactions becomes a single step with no scanning.
This is an O(1) that takes the same amount of time whether you have 10 records or 10,000.

Results of 20 Records compared

I tested both methods across 20 evenly spaced transaction IDs from the dataset. The dictionary lookup search method was 64.6x faster than the linear approach.

```
======================================================
 DSA Search Comparison - 20 searches
======================================================
 ID        Linear (µs)    Dict (µs)   Found?
------------------------------------------------------
 84            11.80µs        1.50µs      Yes
 168           10.40µs        1.40µs      Yes
 252           10.70µs        0.50µs      Yes
 336           14.50µs        0.70µs      Yes
 420           16.30µs        0.40µs      Yes
 504           24.70µs        0.60µs      Yes
 588           30.10µs        0.50µs      Yes
 672           25.20µs        0.40µs      Yes
 756           31.80µs        0.90µs      Yes
 840           34.00µs        0.60µs      Yes
 924           36.80µs        0.40µs      Yes
 1008          73.30µs        1.00µs      Yes
 1092          52.10µs        0.90µs      Yes
 1176          49.80µs        0.70µs      Yes
 1260          52.10µs        0.50µs      Yes
 1344         112.70µs        1.20µs      Yes
 1428         164.20µs        1.50µs      Yes
 1512         145.20µs        1.40µs      Yes
 1596         184.80µs        2.10µs      Yes
 1680         153.00µs        1.90µs      Yes
------------------------------------------------------
 TOTAL       1233.50µs       19.10µs

 → Dictionary lookup was 64.6x faster overall
======================================================
```

Why is dictionary lookup faster?

The dictionary lookup approach is faster because it does not scan. Linear search checks every record from the start until it finds the one needed, so the further the record is, the longer it takes to search it. For instance, in the output for the twenty records compared, record #84 took 4.26µs while record #1680 took 45.37µs. Dictionary lookup works differently; Python dictionaries use a hash table, meaning each value is stored at a memory address derived from its key, so when the user is searching, Python computes the hash and goes straight there in one step, regardless of where the record is, which is why it stayed around 0.2µs the whole time.

Another data structure that can improve the search efficiency

Trie(Prefix Tree) is another method that can make the search much faster. If transaction IDs are string-based like phone-numbers or reference docs, so Trie would be faster than a dictionary. Instead of hashing the entire key, a Trie matches character by character, so the lookup time depends only on the length of the key, not the size of the dataset. Even with millions of records, the search time stays the same as long as the key length doesn't change.

4. Security Analysis — Basic Auth Limitations

4.1 Why Basic Auth Is Weak

HTTP Basic Authentication is simple to implement but carries significant security risks that make it unsuitable for production systems handling sensitive financial data.

Credentials are not encrypted at the transport layer. Basic Auth encodes the username and password in base64, which is trivially reversible — it is encoding, not encryption. Anyone intercepting the request can decode YWRtaW46cGFzc3dvcmQxMjM= back to admin:password123 in under a second. This makes the API completely insecure over plain HTTP, which is how it runs in this project.

Credentials are sent on every request. Unlike token-based systems where a credential is exchanged once for a session token, Basic Auth resends the username and password with every single HTTP request. This multiplies the exposure window — every request is an opportunity for interception.

There is no session management or expiry. Once credentials are known, they remain valid indefinitely until manually changed. There is no concept of a token expiring, a session timing out, or a user being logged out.

Hardcoded credentials are a critical vulnerability. In this implementation, VALID_USERNAME = "admin" and VALID_PASSWORD = "password123" are written directly into the source code. Anyone with access to the repository can read them. Hardcoded credentials cannot be rotated without redeploying the server, and they are frequently exposed through version control history even after being removed.

No protection against brute force. The server has no rate limiting or lockout mechanism. An attacker can attempt thousands of username/password combinations per second without any resistance.

4.2 Stronger Alternatives
JSON Web Tokens (JWT)

JWT is a widely adopted standard for stateless authentication. On login, the server issues a signed token containing claims about the user (such as their ID and role) and an expiry time. The client sends this token in the Authorization: Bearer <token> header on subsequent requests. The server verifies the signature without needing to store session state.
Key advantages over Basic Auth:

Credentials are exchanged once; the token is used for all further requests
Tokens expire automatically, limiting the damage if one is stolen

The payload can carry role and permission information, enabling fine-grained authorization
Tokens are cryptographically signed, so tampering is detectable

For this API, JWTs would require a /login endpoint that validates credentials and returns a token, and each protected endpoint would verify the token's signature and expiry before proceeding.